Pía Amigo Fuentes | Segundo semestre 2026



AST332 Introducción a Machine Learning para Astronomía

Semana 7: Métodos de Ensemble

Métodos de Ensemble

- La idea principal de los métodos de ensemble es **combinar estimadores base** (usualmente, modelos débiles) para obtener un estimador más poderoso (por ejemplo, **más preciso o robusto**).
- Muy a menudo, el estimador base es un **Árbol de Decisión** (o un "stump", que es un árbol con una sola división).
- Dos tipos de ensemble: **Bagging y Boosting**
- Pueden ser usados para clasificación o regresión:
  - **Clasificación**: la combinación suele ser por votación (mayoritaria o ponderada).
  - **Regresión**: se usa el promedio (simple o ponderado) de las predicciones.

Métodos de Ensemble: bagging y boosting

single

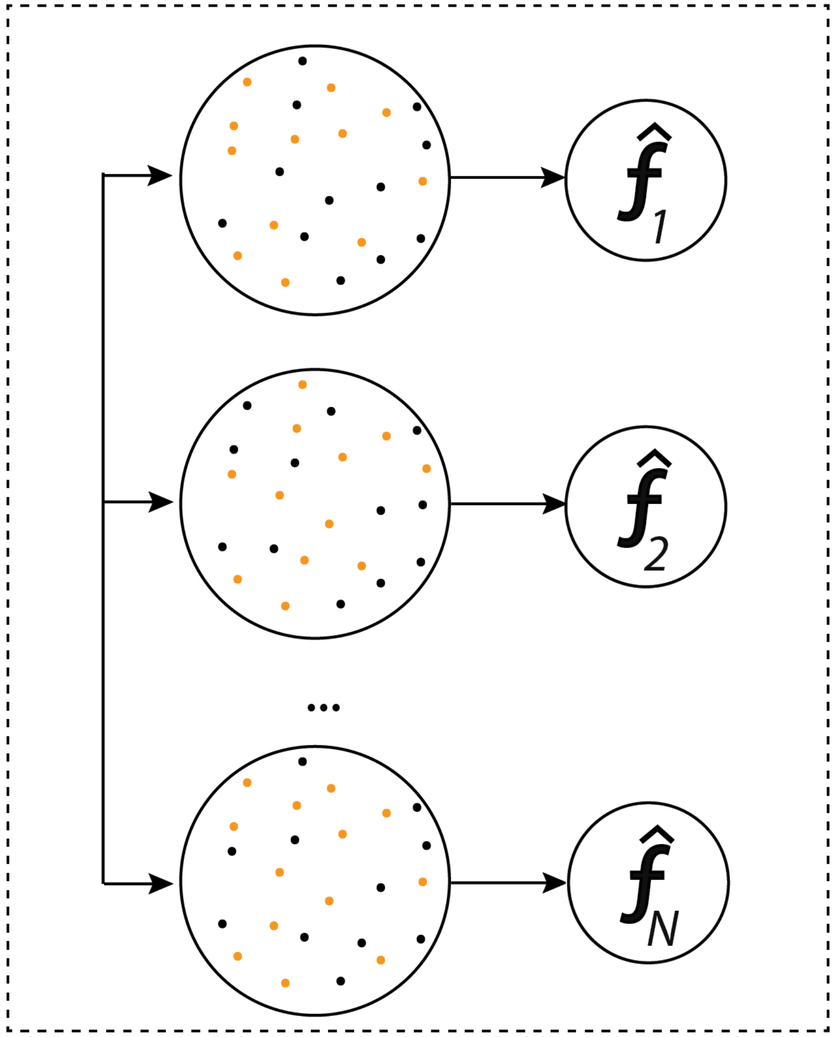
bagging

boosting

Bagging:

- **Los modelos son independientes entre sí**
- **Los modelos tienen el mismo peso**

$$e = \frac{1}{N}\sum_{i=1}^{N} e_i$$

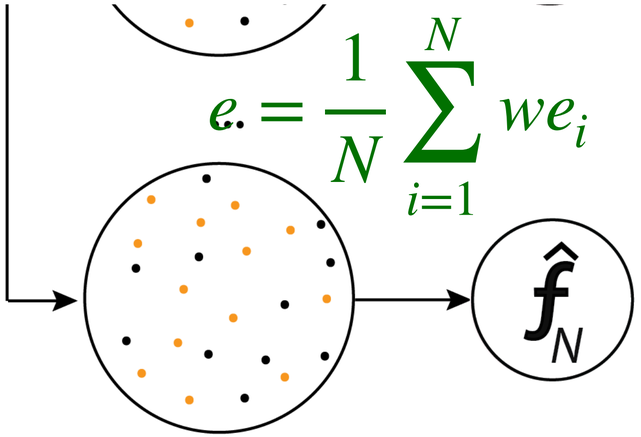


Métodos de Ensemble: bagging y boosting

single

Boosting:

- **el nuevo modelo se ajusta en función de los pasos anteriores (en ejemplos que considera o función de pérdida)**
- **El peso de cada modelo depende de su performance**

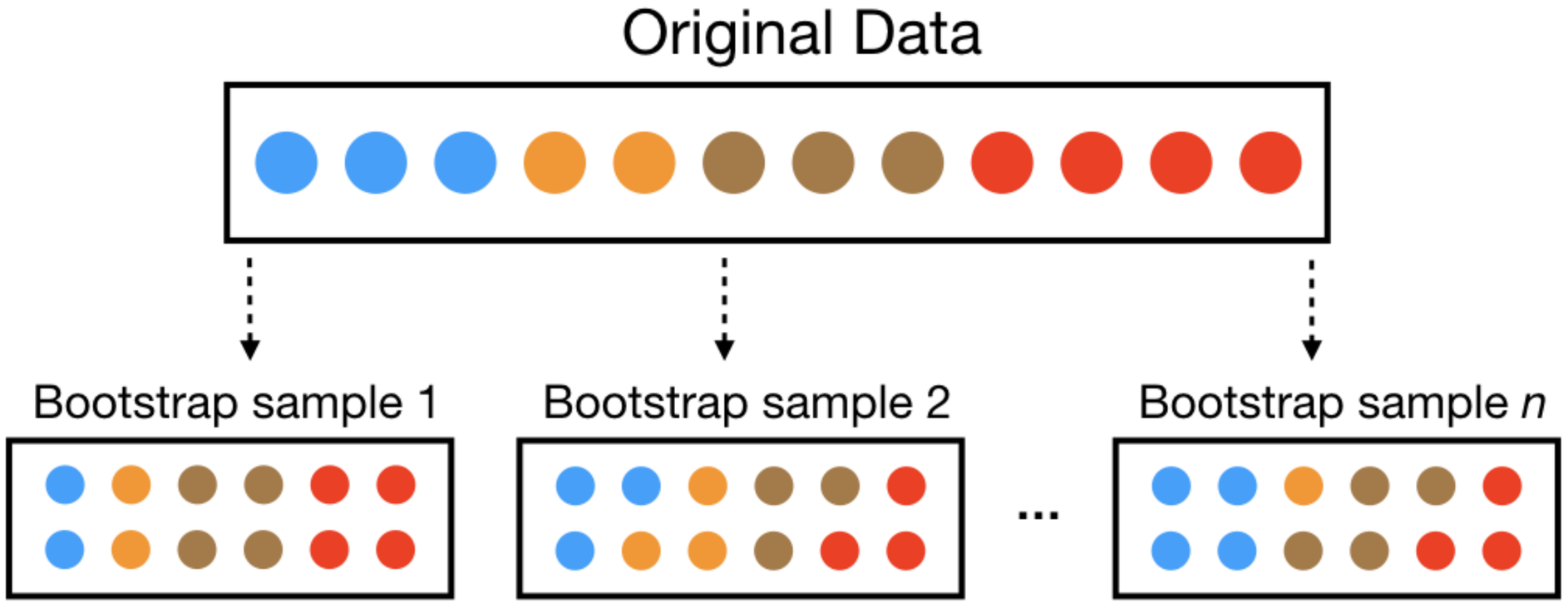


boosting

Bagging (Bootstrap Aggregating)

- El algoritmo entrena varios modelos independientes en subconjuntos aleatorios de datos y combina sus predicciones.
- Resampling: Se construye generando múltiples subconjuntos de los datos de entrenamiento mediante muestreo con reemplazo (bootstrap)
- Cada modelo se entrena de forma independiente en un dataset distinto
- Aggregating: La predicción final se obtiene mediante promedio o votación mayoritaria
- Ejemplo clásico de algoritmo: **Random Forest**

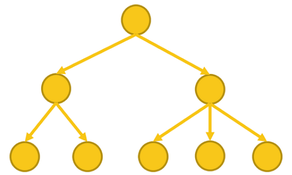
Bootstrap


Original Data
Bootstrap sample 1
Bootstrap sample 2
...
Bootstrap sample *n*

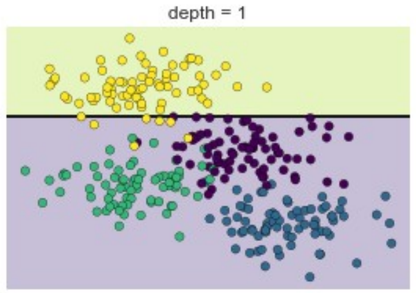
Bagging (Bootstrap Aggregating)

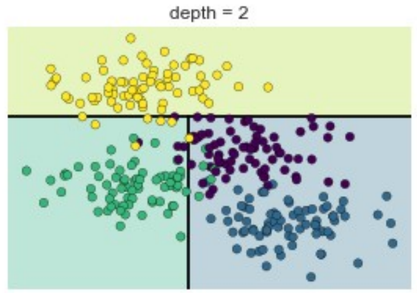
- Bagging entrena muchos modelos independientes $f_b(x)$ usando muestras bootstrap y combina sus predicciones.
- La predicción final es: $\hat{f}(x) = \frac{1}{B}\sum_{b=1}^{B} f_b(x)$
- La varianza del promedio es $\mathbb{V}[\hat{f}(x)] = \frac{1}{B}\mathbb{V}[f_b(x)]$
- Aumentar $B \rightarrow$ disminuye la varianza.
- El bias se mantiene prácticamente igual.
- **En Bagging, todos los modelos son iguales en complejidad, por lo que el sesgo no se ve afectado. Lo que se reduce es la varianza, gracias al promedio de muchos modelos.**

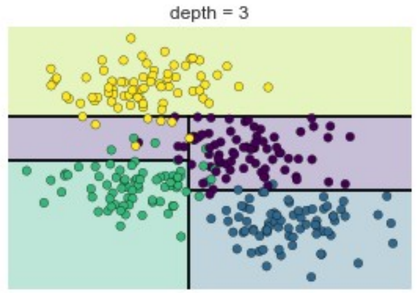
Recap: Árboles de decisión

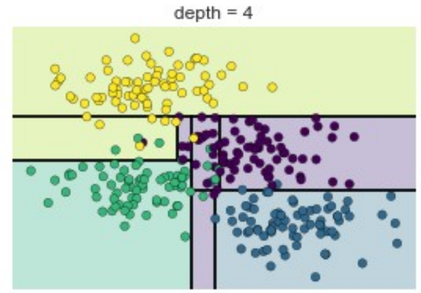


- Rápidos
- Interpretables
- Bajo bias
- Usualmente tienden a sobreajustar (varianza alta)

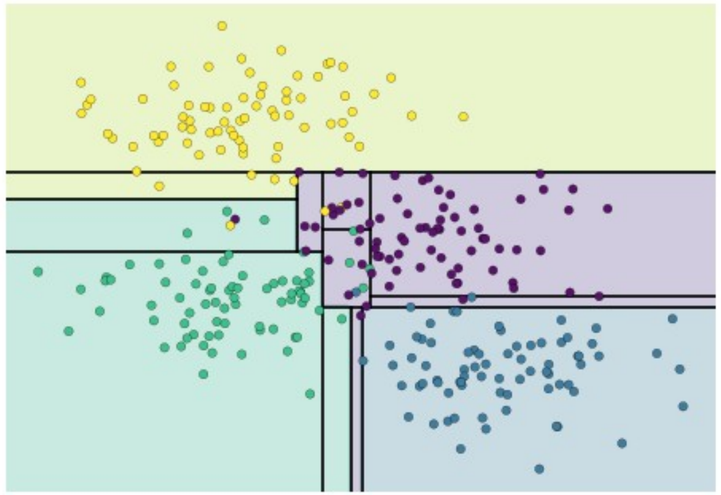








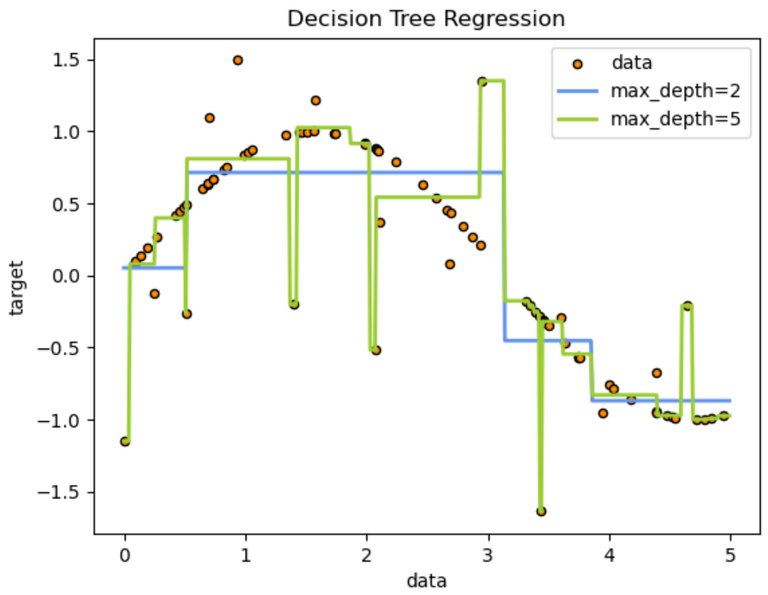
depth=5



Necesitamos “podar” el árbol para deshacernos de las hojas más pequeñas:
`min_impurity_decrease`, `min_samples_split`, `min_samples_leaf`, `max_depth`

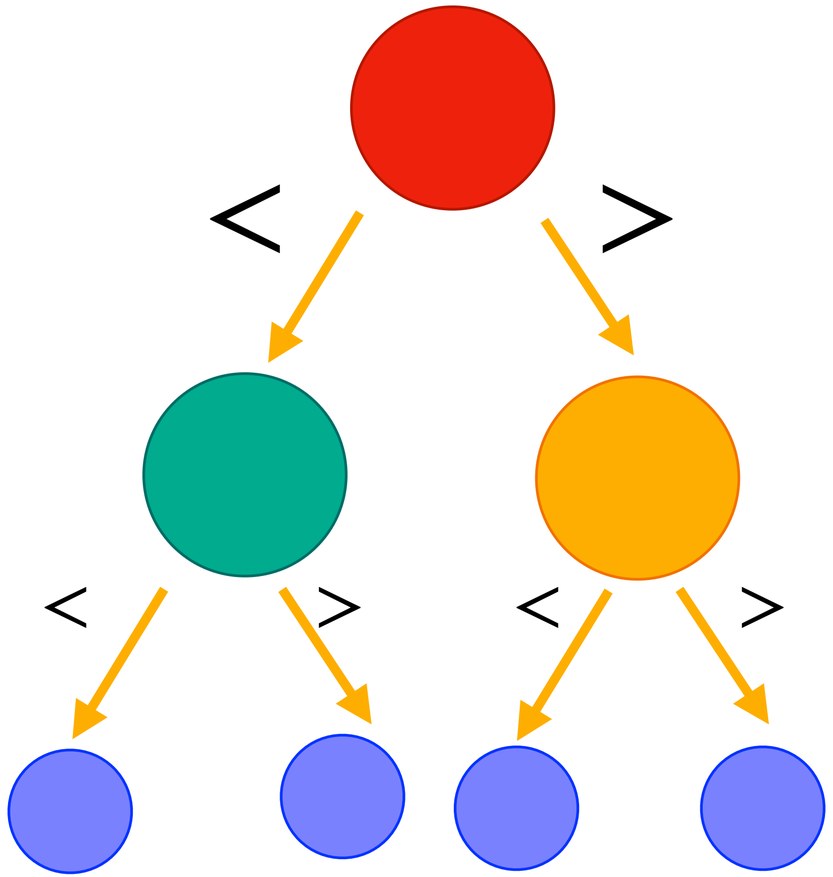
https://jakevdp.github.io/PythonDataScienceHandbook/06.00-figure-code.html#Decision-Tree-Levels

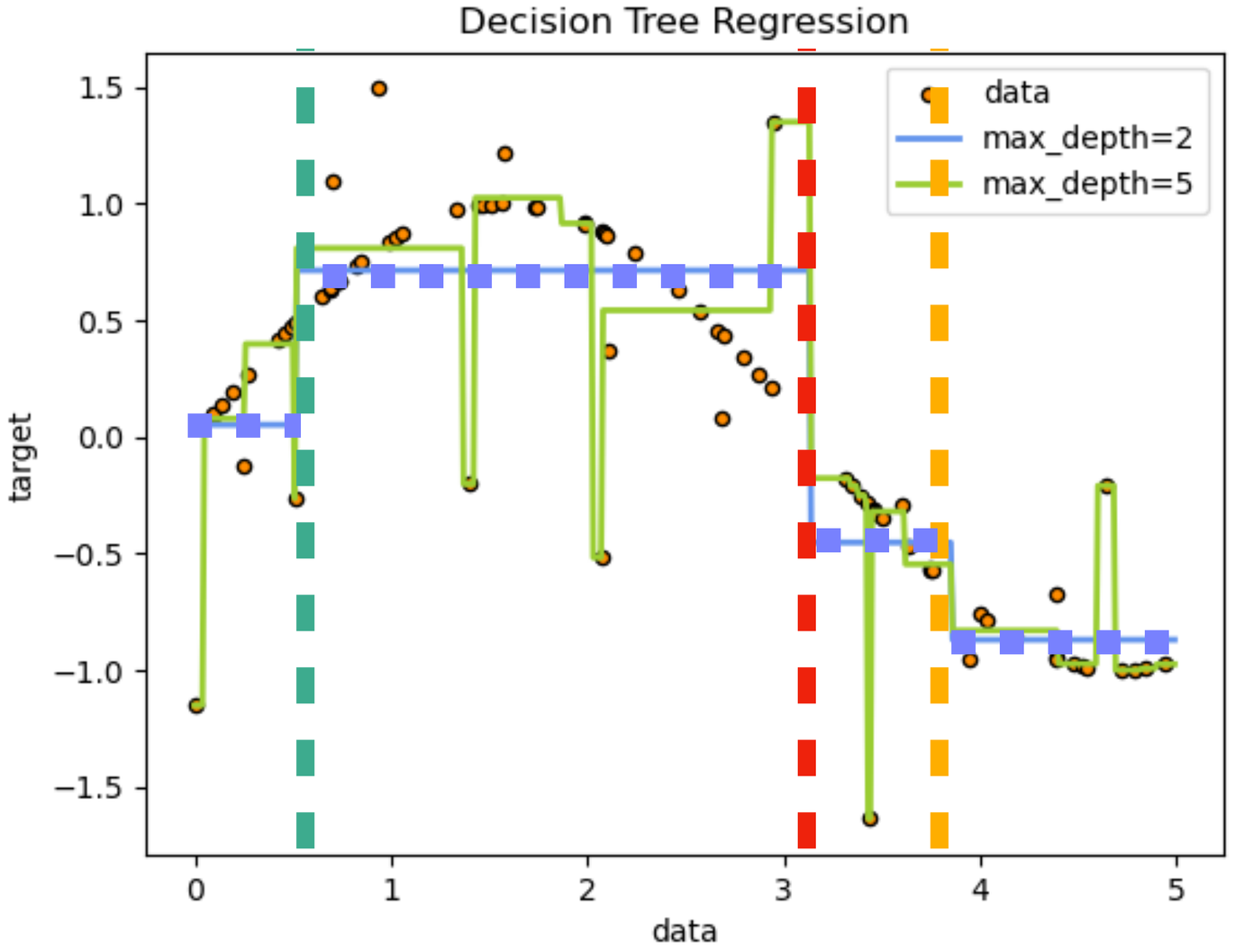
Árboles de decisión para regresión



- En cada nodo, se elige la característica y el umbral que minimicen el MSE en los grupos resultantes.
- La división se realiza recursivamente hasta cumplir un criterio para detenerse (ej., profundidad máxima, mínimo de observaciones por nodo).
- Para cada posible división:
  - Se calcula el promedio del target y en cada grupo.
  - Se mide el MSE como el promedio de las diferencias cuadradas entre los valores de y y el promedio del grupo.
- El split que minimiza el MSE total (promedio ponderado) es seleccionado.
- La predicción para una nueva observación es el valor promedio de y en la hoja donde cae la observación.

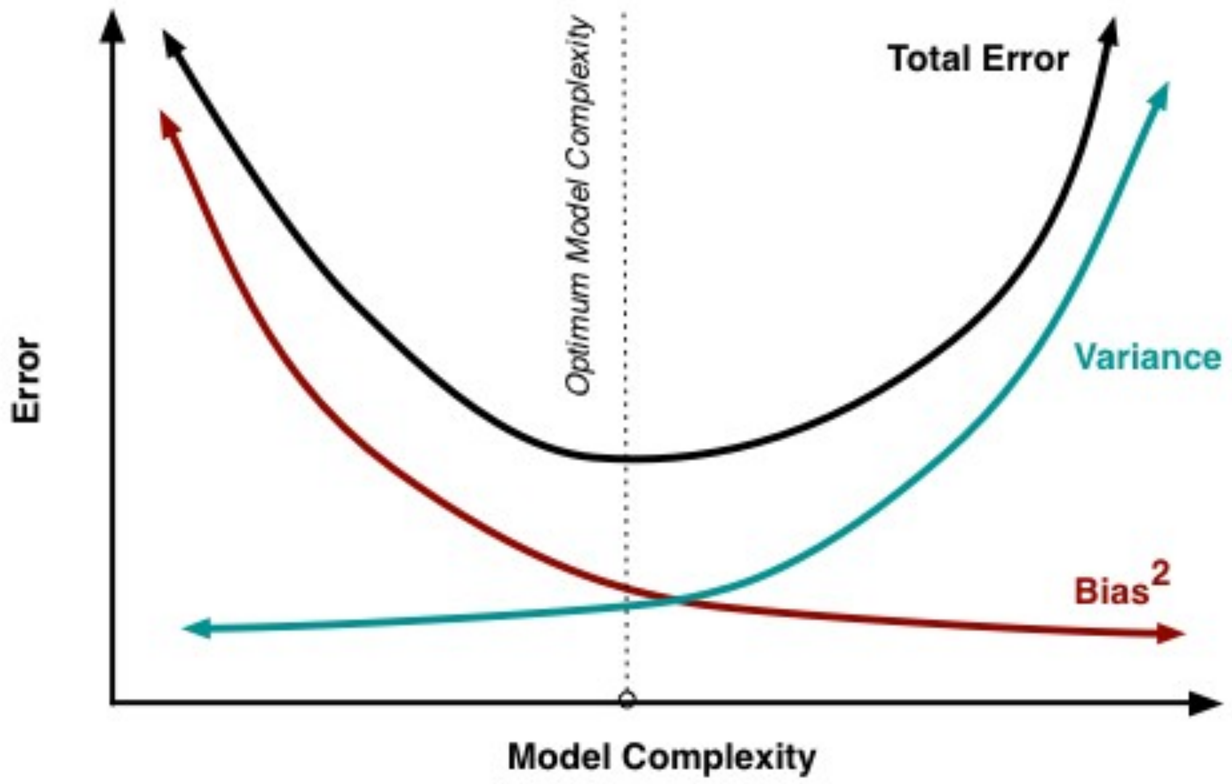
Árboles de decisión para regresión





Bias-Variance decomposition para ensemble de árboles de decisión

$$\mathrm{MSE}(x) = \mathrm{bias}^2(x) + \mathrm{variance}(x) + \mathcal{N}(x)$$



Si estamos dispuestos a aceptar un pequeño aumento en el bias, podemos reducir la varianza combinando árboles aleatorizados y aún así obtener un modelo mejorado (es decir, con menores errores en los datos de prueba).

Bias-Variance decomposition para ensemble de árboles de decisión

$$\mathrm{MSE(x)} = \mathrm{bias}^2\mathrm{(x)} + \mathrm{var(x)} + \mathcal{N}\mathrm{(x)}$$

$$\mathbb{E}\left[\left(\hat{y}(x) - y(x)\right)^2\right] = \underbrace{\left(\mathbb{E}[\hat{y}(x)] - y(x)\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\left[\left(\hat{y}(x) - \mathbb{E}[\hat{y}(x)]\right)^2\right]}_{\text{Varianza}} + \underbrace{\mathcal{N}}_{\text{Ruido irreducible}}$$

$$\mathrm{Var}_{\mathrm{ensemble}}(x) = \rho(x)\sigma^2(x) + \frac{1-\rho(x)}{M}\sigma^2(x)$$

donde $\sigma^2(x)$ es la varianza de la predicción de un solo árbol, M es el número de árboles, y $\rho(x)$ es el coeficiente de correlación de Pearson entre las predicciones de dos árboles aleatorizados construidos sobre el mismo conjunto de aprendizaje.

Bias-Variance decomposition para ensemble de árboles de decisión

El bias es del conjunto M de árboles es idéntico al de sólo un árbol, si cada árbol es generado por el mismo mecanismo. Cada árbol está limitado por el tipo de modelo que es (por ejemplo, un árbol poco profundo) y los elementos de aleatorización. Este bias es el error sistemático que comete el árbol al no capturar todos los patrones complejos de los datos.

La esperanza de un promedio es el promedio de las esperanzas $\mathbb{E}\left[\frac{1}{M}\sum_{m=1}^{M}\hat{y}_m(x)\right] = \frac{1}{M}\sum_{m=1}^{M}\mathbb{E}[\hat{y}_m(x)]$

Si todos los árboles siguen la misma distribución: $\mathbb{E}[\hat{y}_1(x)] = \mathbb{E}[\hat{y}_2(x)] = \ldots = \mathbb{E}[\hat{y}_M(x)]$

$$\frac{1}{M}\sum_{m=1}^{M}\mathbb{E}[\hat{y}_m(x)] = \mathbb{E}[\hat{y}_1(x)]$$

$$\Longrightarrow \quad \mathbb{E}\left[\frac{1}{M}\sum_{m=1}^{M}\hat{y}_m(x)\right] = \mathbb{E}[\hat{y}_1(x)]$$

La esperanza del ensemble es la misma que de un solo árbol

Bias-Variance decomposition para ensemble de árboles de decisión

El bias es del conjunto M de árboles es idéntico al de sólo un árbol. Cada árbol está limitado por el tipo de modelo que es (por ejemplo, un árbol poco profundo) y los elementos de aleatorización. Este bias es el error sistemático que comete el árbol al no capturar todos los patrones complejos de los datos.

$$\mathrm{Var}_{\mathrm{ensemble}}(x) = \rho(x)\sigma^2(x) + \frac{1-\rho(x)}{M}\sigma^2(x)$$

Si $\rho(x) \approx 1$ $\Longrightarrow$ Los árboles están **muy correlacionados** y, por lo tanto, hacen predicciones muy similares para la observación en x, esto implica que los errores que cometen los árboles están alineados entre sí; todos los árboles "piensan" lo mismo y sus errores no se cancelan cuando se combinan. El término $\rho(x)\sigma^2(x)$ domina en la fórmula de la varianza del ensemble, y la varianza del conjunto será cercana a la varianza de un solo árbol. En otras palabras, **no se reduce mucho la varianza al combinar árboles, ya que los errores son muy similares.**

Bias-Variance decomposition para ensemble de árboles de decisión

El bias es del conjunto M de árboles es idéntico al de sólo un árbol. Cada árbol está limitado por el tipo de modelo que es (por ejemplo, un árbol poco profundo) y los elementos de aleatorización. Este bias es el error sistemático que comete el árbol al no capturar todos los patrones complejos de los datos.

$$\mathrm{Var}_{\mathrm{ensemble}}(x) = \rho(x)\sigma^2(x) + \frac{1-\rho(x)}{M}\sigma^2(x)$$

Si $\rho(x) \approx 0 \implies$ Los árboles están **poco correlacionados** y sus predicciones para x son menos similares. Esto significa que cada árbol comete errores de manera diferente; los errores individuales no están alineados y, al combinar las predicciones, estos errores tienden a cancelarse entre sí. En este caso, el término $\frac{1-\rho(x)}{M}\sigma^2(x)$ empieza a dominar en la fórmula, y la **varianza del conjunto puede reducirse considerablemente al aumentar el número de árboles** M

Bias-Variance decomposition para ensemble de árboles de decisión

$$\mathrm{Var}_{\mathrm{ensemble}}(x) = \rho(x)\sigma^2(x) + \frac{1-\rho(x)}{M}\sigma^2(x)$$

Si $M \to \infty$ (aumento el número de árboles), $\mathrm{var}(x) \to \rho(x)\sigma^2(x)$

- Para aleatorización fuerte: $\rho(x) \to 0,\ \mathrm{var}(x) \to 0$
- Para aleatorización débil: $\rho(x) \to 1,\ \mathrm{var}(x) \to \sigma^2(x)$

Una aleatorización más fuerte puede aumentar el bias, porque cada árbol ve menos información en cada split o queda más restringido. Sin embargo, también puede reducir la correlación entre árboles, lo que disminuye la varianza del ensemble. **La clave del problema es encontrar el equilibrio adecuado (recuerde que reducir la varianza no garantiza mejores puntajes en la prueba!)**

Técnicas de aleatorización

- Bootstrap samples (con reemplazo)

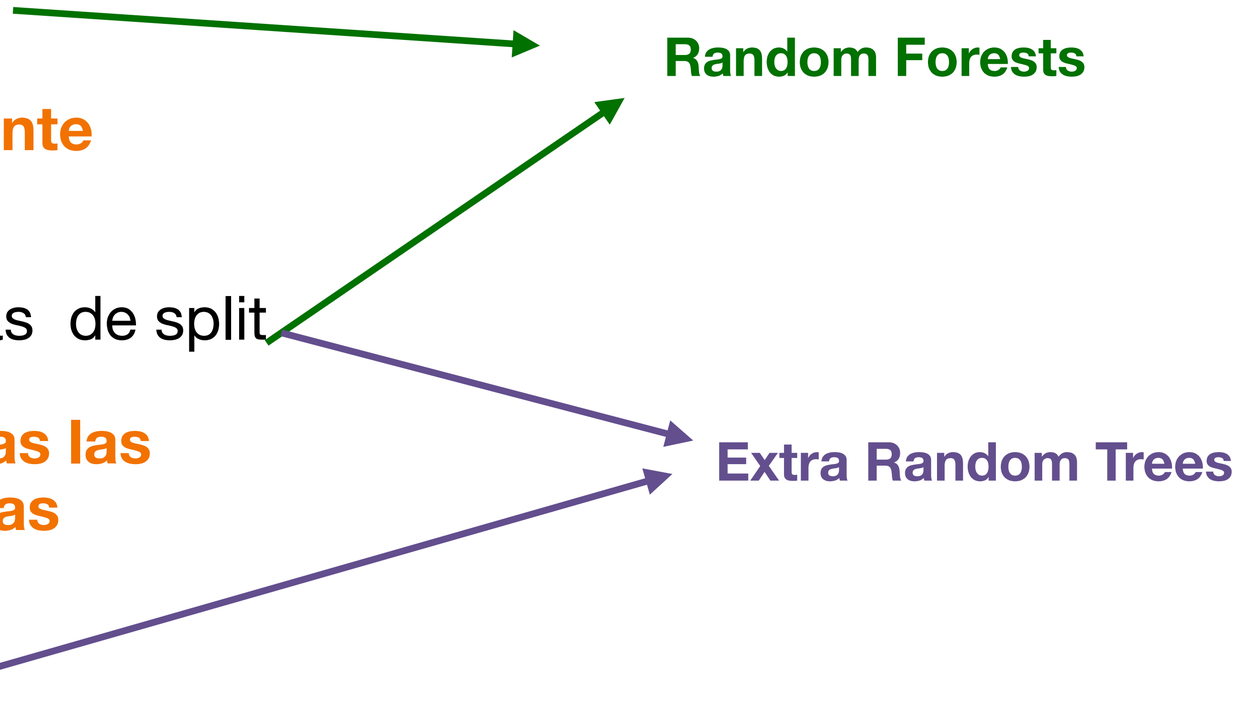
  Elige boostrap samples levemente distintos

- Selección aleatoria de K características de split

  Elige el mejor split no sobre todas las características, sino sobre algunas aleatorias

- Selección aleatoria del split

  El valor umbral para la división se selecciona aleatoriamente



Hiperparámetros del modelo

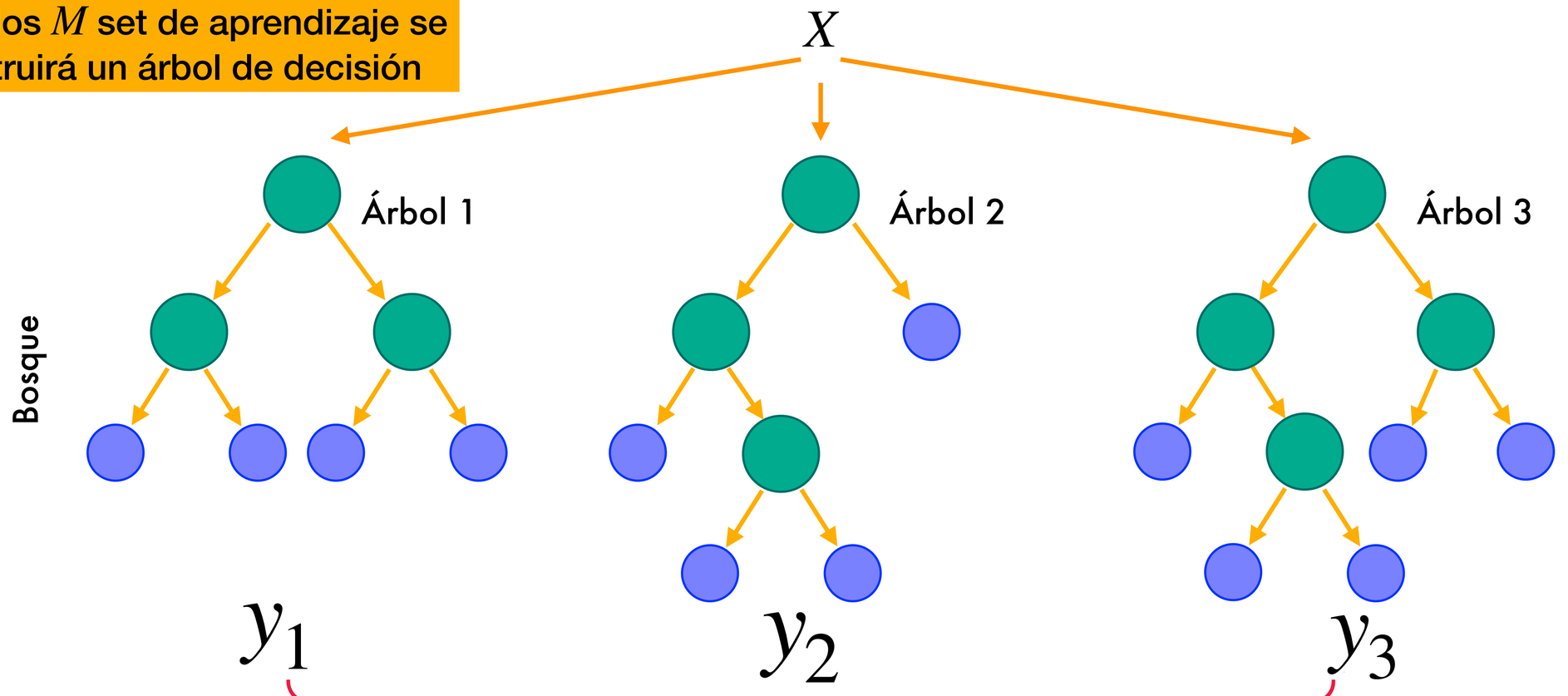
Random Forests

(Bagging)

El set de datos original es replicado M veces usando bootstrap sampling con reemplazo.

Sobre los M set de aprendizaje se construirá un árbol de decisión

Cuando se crean los árboles de decisión, la características que participan en la selección del split óptimo es un subset aleatorio de las características totales



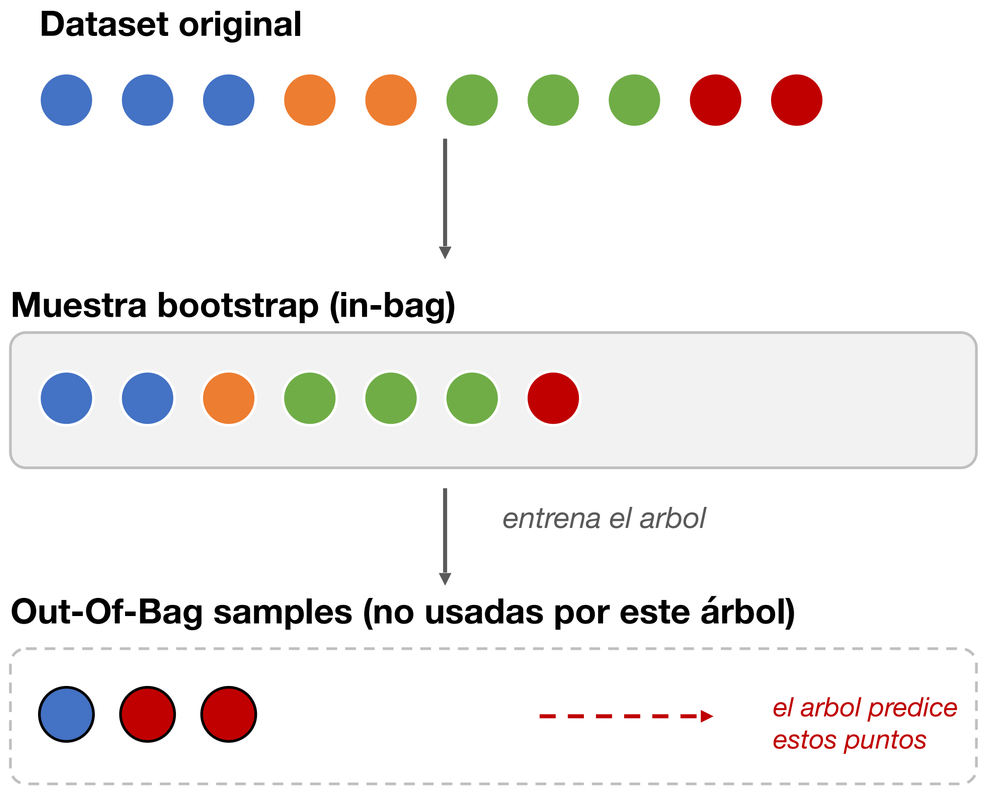
$$y = < y_1, y_2, y_3 >$$

Los árboles de decisión se construyen independientemente, y cada uno nos da una predicción por cada elemento del set de entrenamiento

La predicción final es simplemente el promedio de todas las predicciones (para problemas de regresión) o el voto mayoritario (para problemas de clasificación)

Out-of-Bag (OOB) Error

- Cada árbol se entrena con una muestra bootstrap (con reemplazo).
- En promedio, ~37% de las observaciones quedan fuera de esa muestra para un árbol dado (las OOB samples de ese árbol).
- Cada observación es evaluada solo por los árboles que no la usaron para entrenar.
- Promediando esas predicciones se obtiene el error OOB: validación integrada, sin gastar datos en un set aparte.
- En sklearn: `oob_score=True` en RandomForestRegressor / Classifier



Error OOB
= comparar la predicción del árbol sobre sus propias OOB samples con el valor real

Métodos de Boosting

- Los métodos de boosting trabajan secuencialmente:
  - Se crea un árbol pequeño (stump) para hacer predicciones, y en el siguiente paso, el algoritmo se enfoca en predecir correctamente los ejemplos más problemáticos, y el proceso se aplica iterativamente
  - Dos formas principales:
    - Actualizar los pesos en cada ejemplo (AdaBoost)
    - Actualizar la función de pérdida (Gradient Boosting methods)

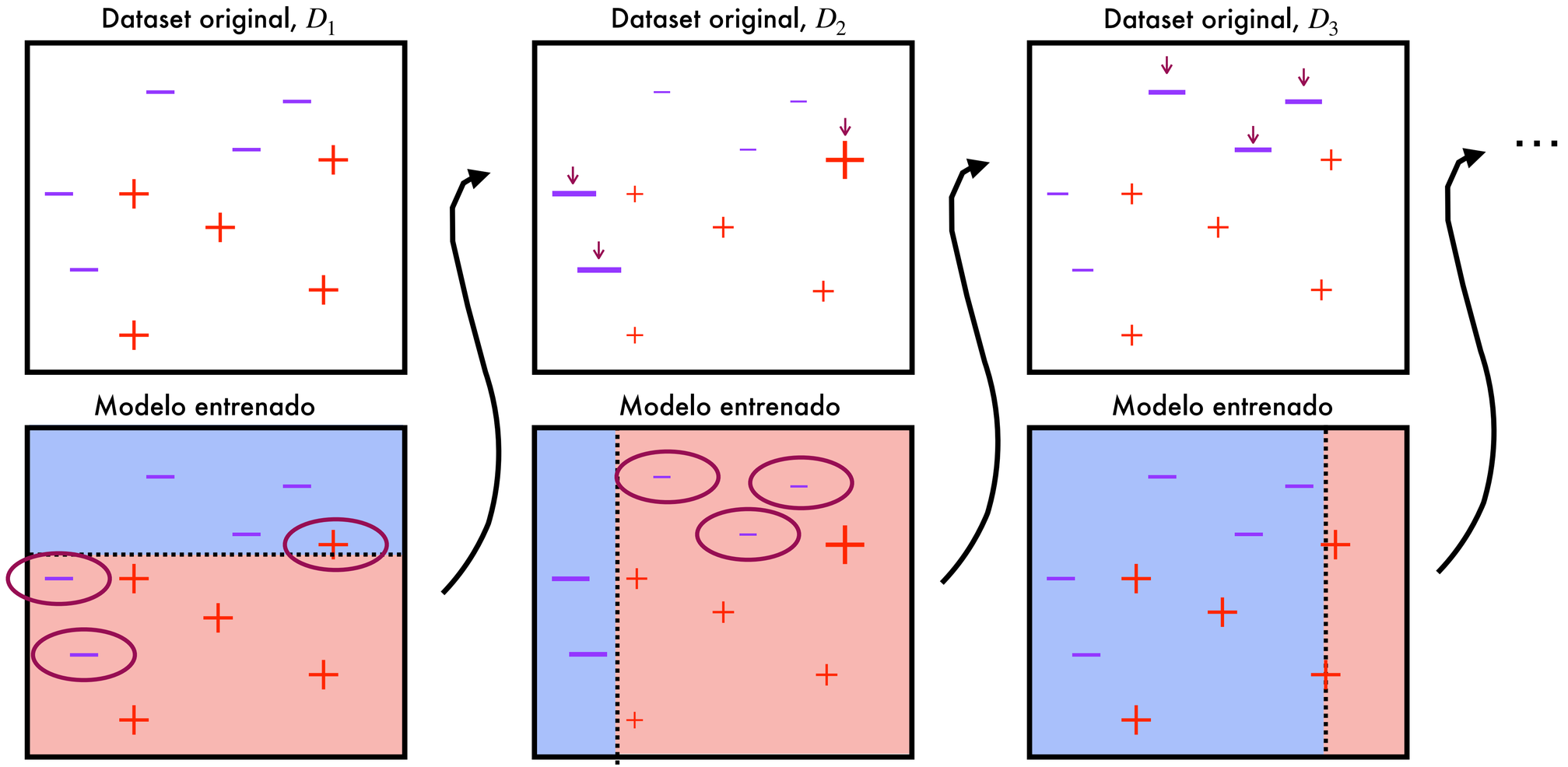
Boosting

- Boosting combina modelos débiles $f_m(x)$ en secuencia, donde cada modelo corrige los errores del anterior.
- Predicción final: $F_M(x) = \sum_{m=1}^{M} \gamma_m f_m(x)$ (M es el número de modelos, γ_m es el peso de cada modelo, de acuerdo a qué tan bien le fue con los errores del anterior)
- Reducción del bias: $\text{Bias}^2(F_M) < \text{Bias}^2(f_1)$
- **Boosting reduce el sesgo porque cada modelo aprende de los errores del anterior, acercando progresivamente las predicciones al valor verdadero.**

AdaBoost (Boosting)

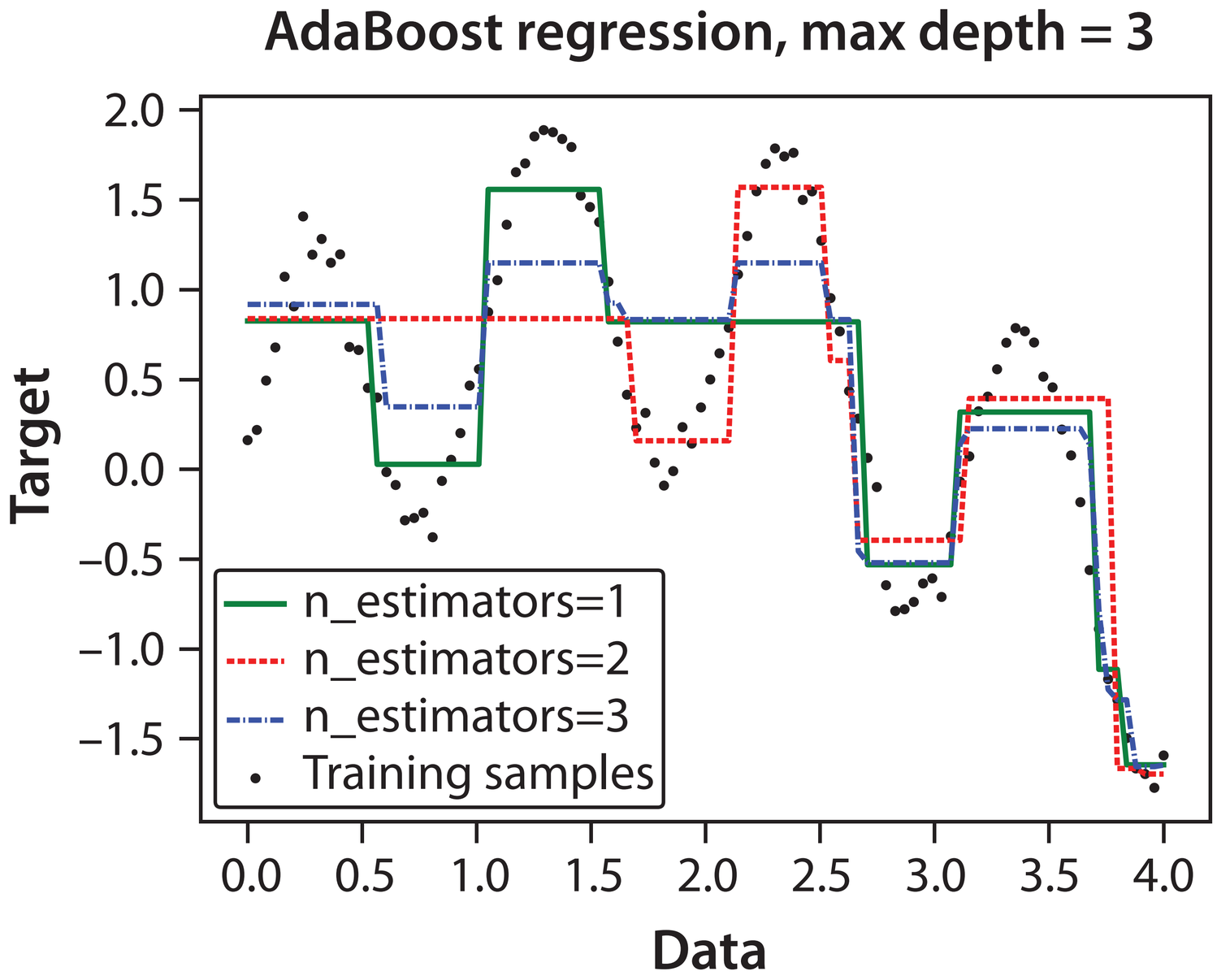
- Es un método de boosting donde inicialmente se le asigna el mismo peso a todos los datos
- Se entrena un modelo débil, como un árbol de decisión de una sola división (un "stump").
- **AdaBoost evalúa las predicciones y aumenta el peso de las observaciones que fueron mal clasificadas, mientras que reduce el peso de las observaciones clasificadas correctamente.**
- Este proceso asegura que el siguiente modelo se enfoque más en las observaciones difíciles o mal clasificadas.
- **AdaBoost calcula un peso para el modelo en función de su precisión. Los modelos con mejor precisión tendrán mayor influencia en la predicción final.**
- La predicción final es una combinación ponderada de las predicciones de todos los modelos, donde los pesos se asignan según el rendimiento de cada modelo.
- Limitación: Sensible a outliers, ya que les da más peso en cada iteración.

AdaBoost


Dataset original, D_1
Dataset original, D_2
Dataset original, D_3
Modelo entrenado
Modelo entrenado
Modelo entrenado
...

AdaBoost

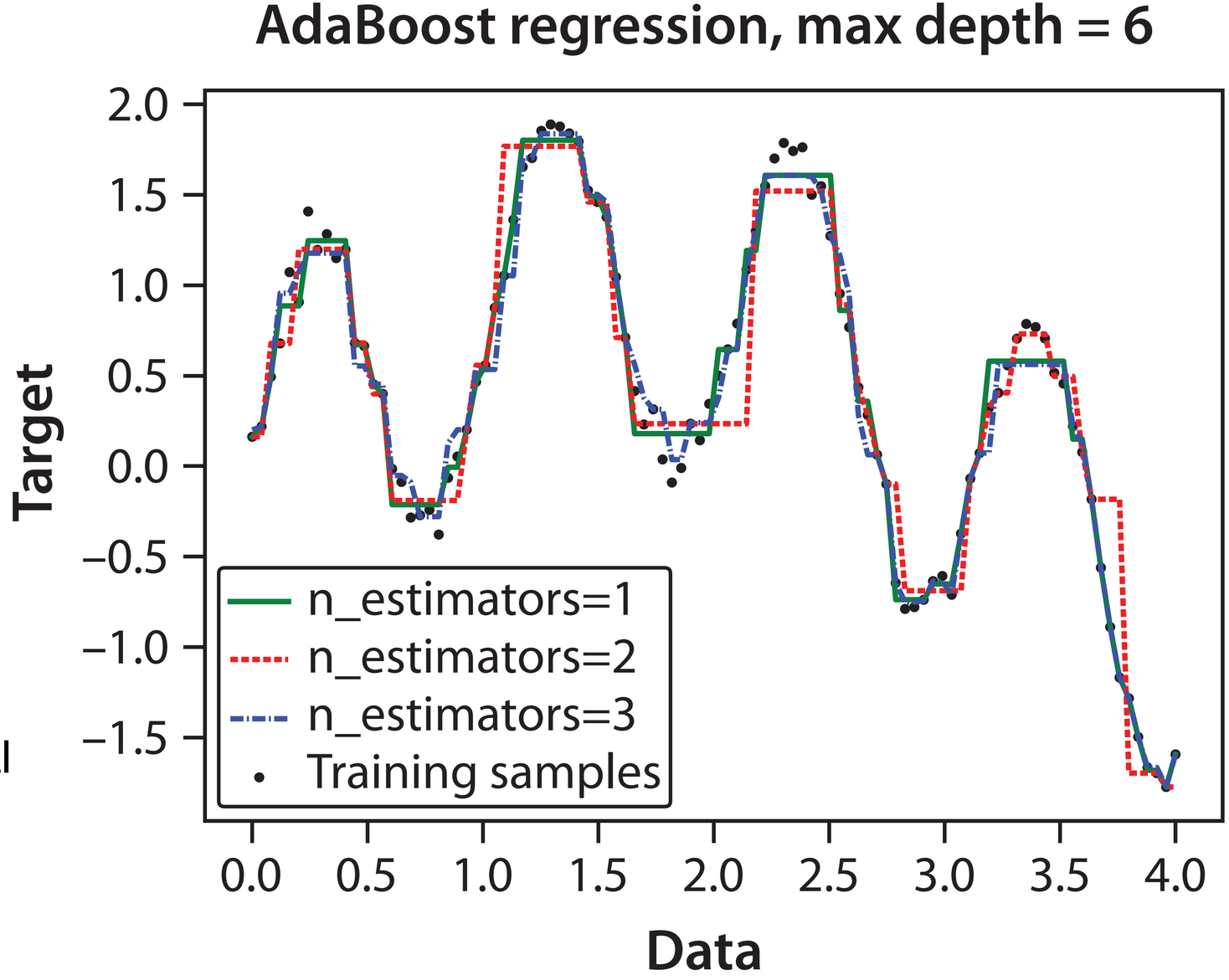
Modelo base no es suficientemente poderoso



AdaBoost

Modelo base es suficientemente poderoso

AdaBoost necesita un modelo base con suficiente capacidad propia. Reponderar las observaciones difíciles no le da al modelo más capacidad para representar formas complejas.



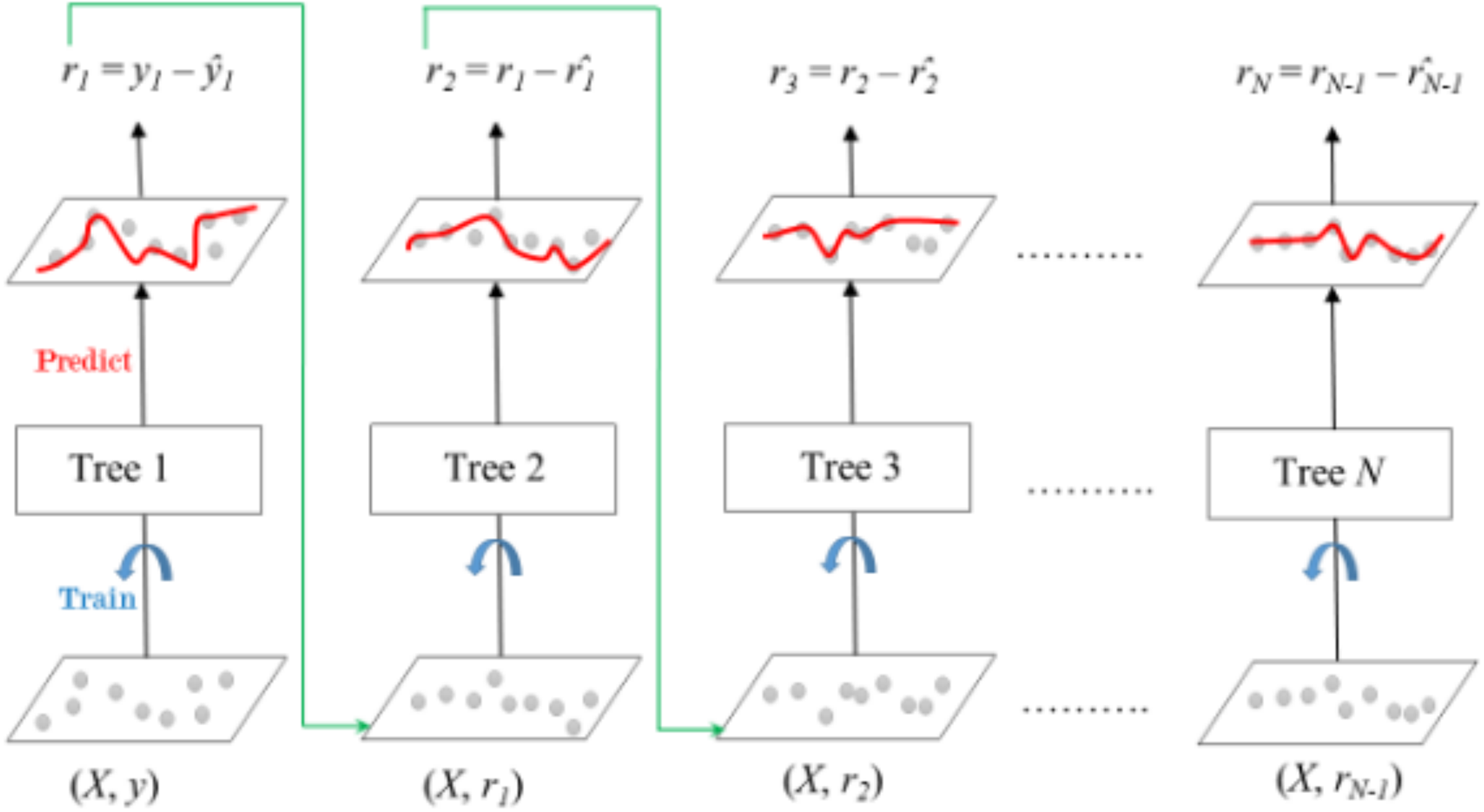
Gradient Boosted Trees (Boosting)

- Comienza con una predicción inicial, que suele ser el promedio de los valores objetivo (para regresión).
- Se calcula el error residual, que es la diferencia entre las predicciones actuales y los valores verdaderos.
- **En cada iteración, se entrena un nuevo modelo débil que intenta predecir estos errores residuales.**
- Este modelo actúa como una "corrección" a las predicciones anteriores, ajustándose para reducir el error total.
- El modelo final es una suma de los modelos ajustados en cada paso, multiplicados por un parámetro de aprendizaje (o "learning rate") que controla la contribución de cada modelo.

$$F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} f_m(x)$$

- Esto permite un ajuste controlado y evita sobreajuste.
- La predicción final se obtiene sumando todas las correcciones generadas en cada iteración.

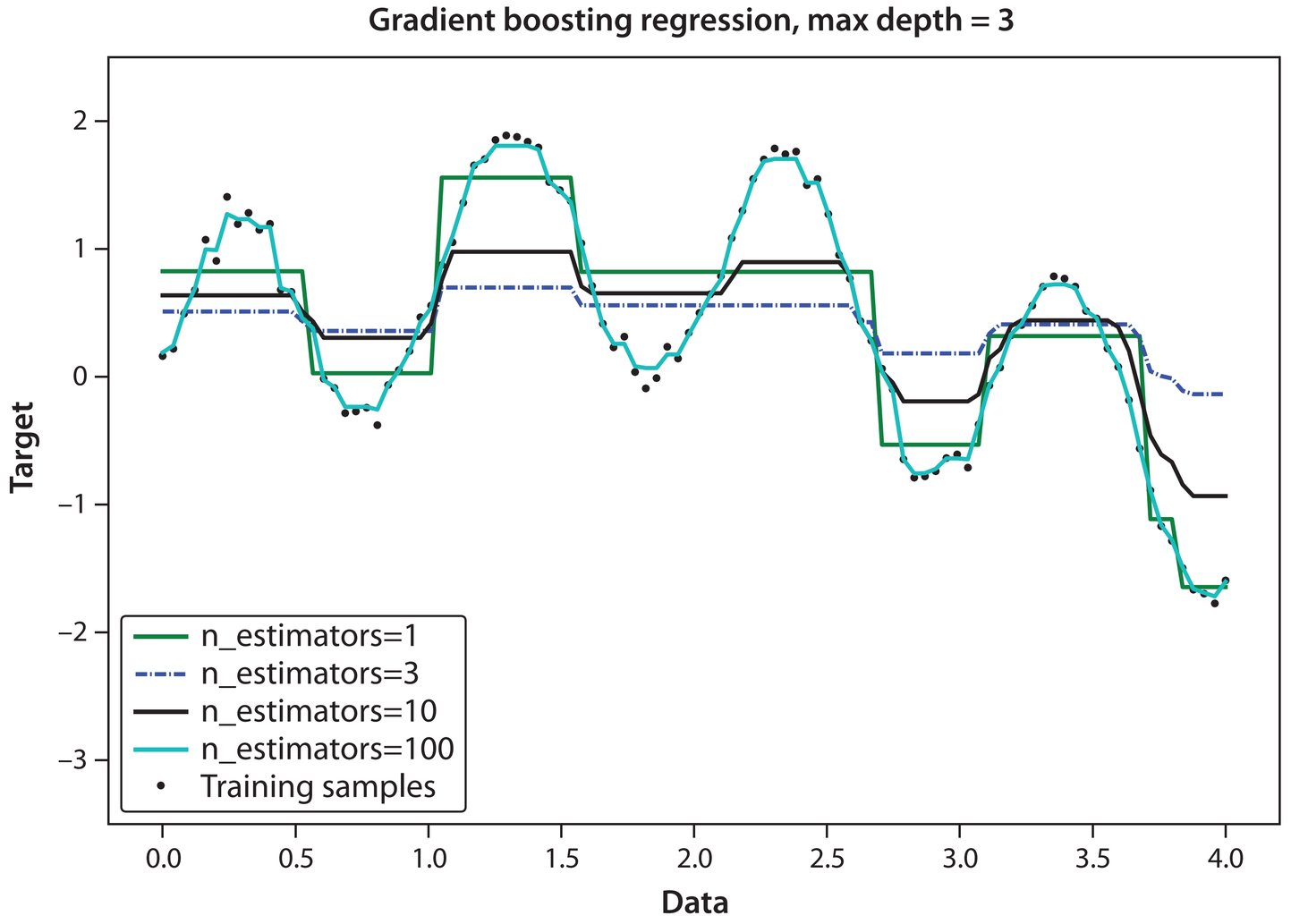
Gradient Boosted Trees



Gradient Boosted Trees

modelos base simple y confiar en el número de estimadores (y el learning rate) para construir la complejidad necesaria

Como cada árbol nuevo corrige el residual del paso anterior (no reajusta el problema completo), sumar suficientes correcciones pequeñas permite converger a un ajuste fino de la función, sin necesitar árboles individuales más profundos



Gradient Boosted Trees

- **Limitación**: Puede ser más lento y propenso al sobreajuste si no se ajusta cuidadosamente el parámetro de aprendizaje y la cantidad de modelos

- Podemos incluir técnicas de **regularización** para mitigar el sobreajuste

    - **Shrinkage**: Elige un valor pequeño (<1) para el coeficiente η también conocido en sklearn como learning rate, el cual regula la contribución de cada nueva corrección al modelo actual. Un GBM con valores menores de tardará más en alcanzar la misma precisión; hay una compensación entre el número de iteraciones y el learning rate. Si dejáramos $\eta = 1$, el modelo corregiría todo el residual, tendiendo a sobreajuste

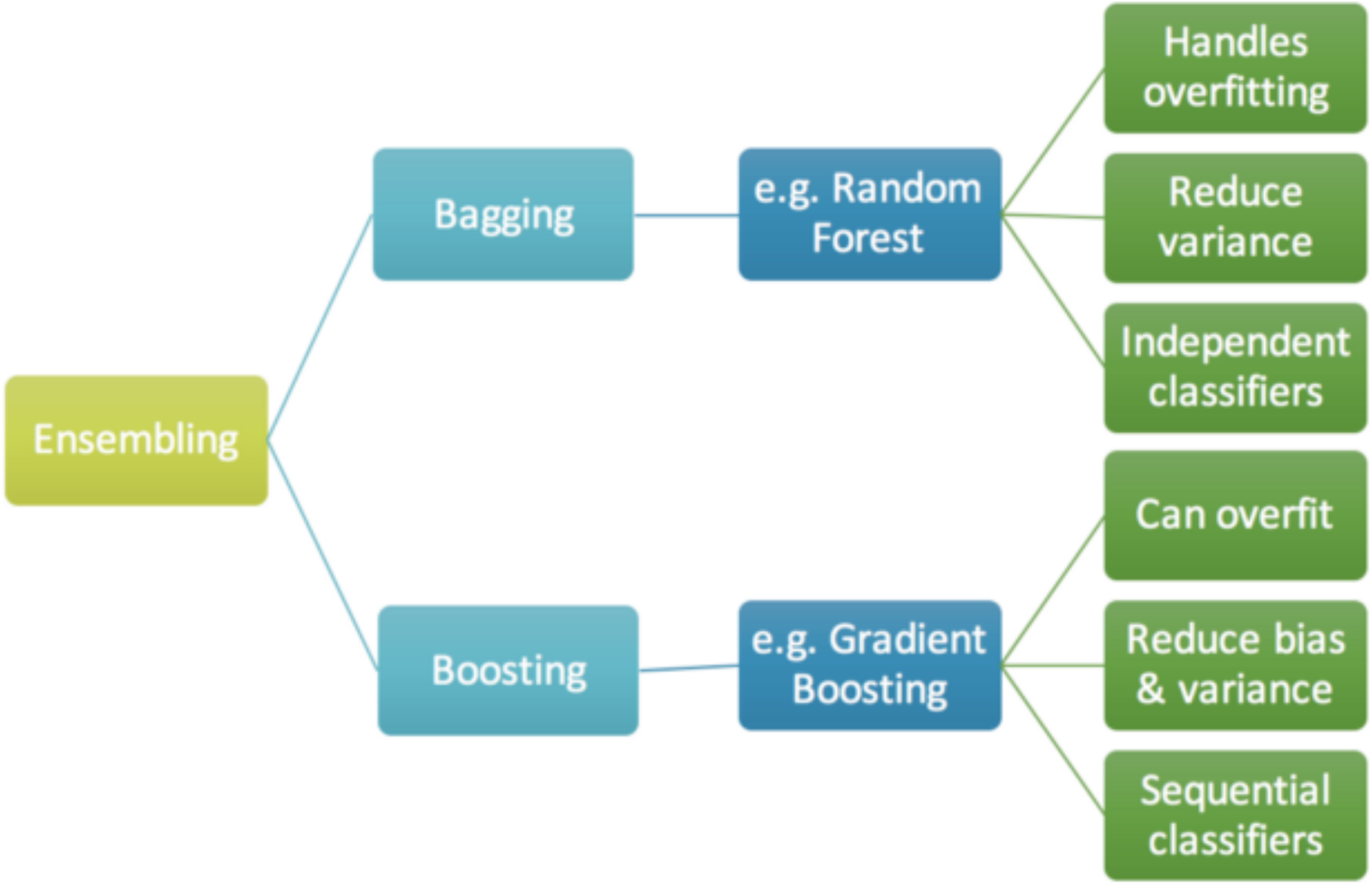
    - **Subsampling**: Selecciona una fracción aleatoria de los datos (con o sin reemplazo) para ajustar cada paso. Cuanto menor sea la fracción, más fuerte será la regularización.

    - **Selección aleatoria de características**: Usa solo un subconjunto aleatorio de características en cada paso, similar a cómo lo utiliza Random Forests para reducir la varianza del estimador individual.

Para qué se usan los métodos de bagging y boosting

- Problemas tanto de clasificación como de regresión
- Siempre puedes convertir un problema de clasificación en uno de regresión prediciendo la probabilidad de que un objeto pertenezca a una cierta clase
- Son conjuntos de algoritmos de propósito general, generalmente rápidos y precisos
- Para datos tabulares (donde no necesitas aprender una representación, es decir, estás satisfecho con tu espacio de características original), a menudo es el mejor método disponible

Pequeño resumen



* con regularización

Feature Importance

Una característica interesante de los métodos de ensemble es que es fácil construir una comprensión de **qué características contienen la mayor parte de la información relevante.**

Típicamente, las variables se clasifican según dos criterios:

- La disminución media de la impureza (MDI): sumando las reducciones de impureza en todos los nodos de los árboles donde aparece la variable (Breiman et al., 1984)

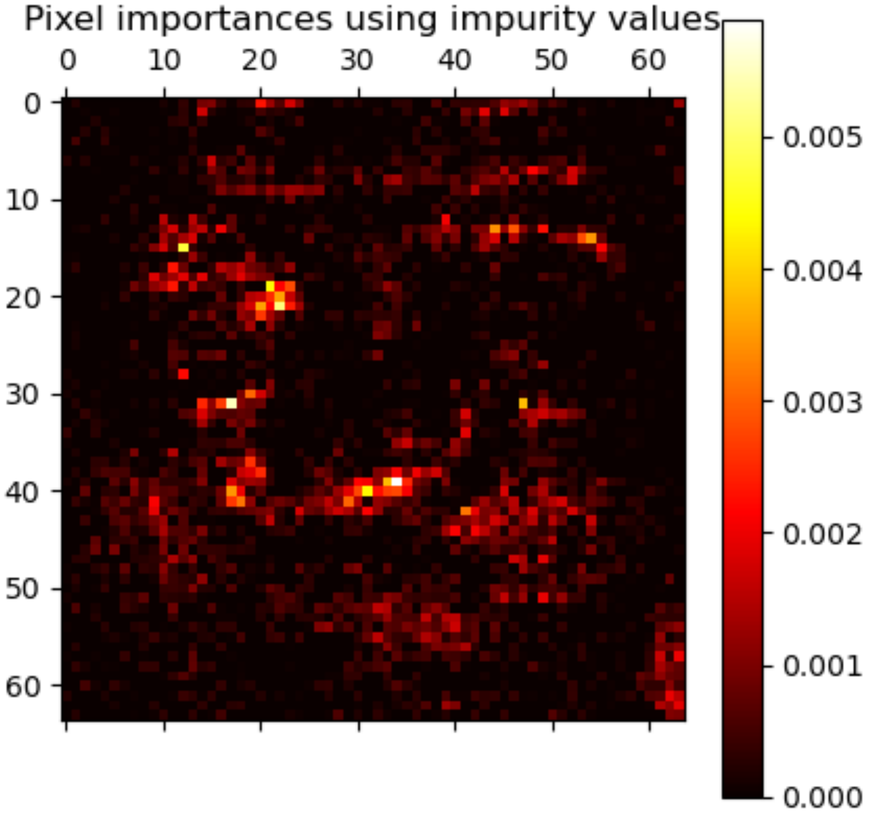
- La disminución media de la exactitud (MDA): midiendo la reducción de exactitud en las muestras fuera de bolsa (out-of-bag) cuando los valores de la variable se permutan aleatoriamente (Breiman, 2001).

MDI mide cuánto utiliza el árbol una variable para reducir impureza, Permutation Importance mide cuánto empeora la capacidad predictiva del modelo cuando perturbamos esa variable.

Feature Importance

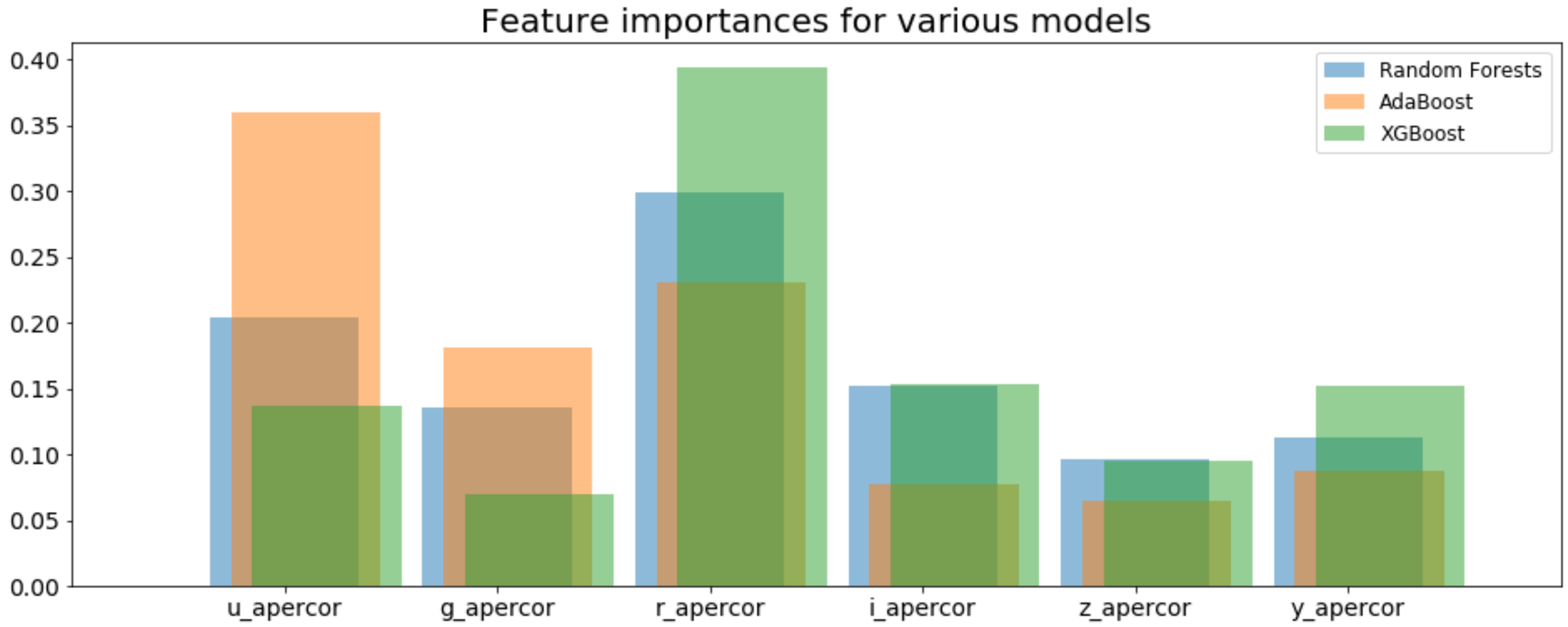
Una variable es más importante si conduce a una gran Disminución Media de Impureza o Disminución Media de Exactitud (en otras palabras, tiene un gran impacto en el rendimiento).

El ranking de características se puede usar para obtener una comprensión de los datos y para reducir el tamaño de un conjunto de datos (por ejemplo, se puede clasificar el conjunto de datos original y seleccionar las primeras x de entre y características, utilizando la pérdida de información como guía).



https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_importances_faces.html

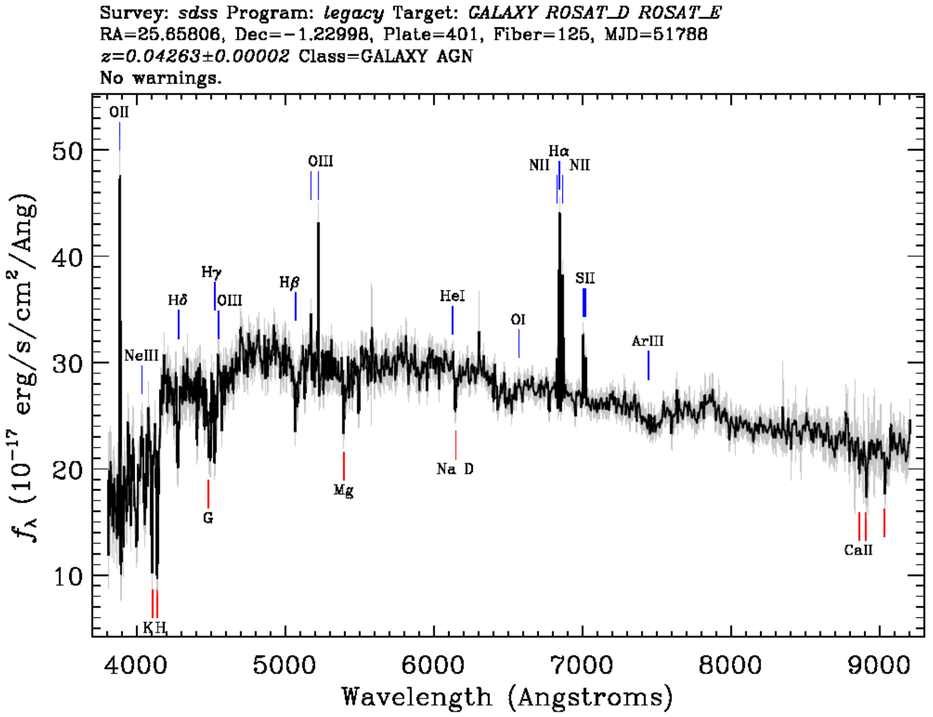
Feature Importance depende del algoritmo!



Puedes usar estos métodos para descartar características poco importantes de manera bastante segura, pero ojo con el ranking!

Ejemplo de aplicación: Redshift fotométrico

- Un espectro es un gráfico de alta resolución de brillo frente a longitud de onda.
- Para galaxias que están más lejos, el espectro se estira (todas las longitudes de onda son más largas).
- Los espectros contienen picos y caídas, que corresponden a transiciones conocidas en átomos básicos (por ejemplo, H, O).
- Si puedo identificar las líneas de emisión que veo (a partir de la estructura – ¡una sola no es suficiente!), puedo calcular la cantidad de estiramiento, que es 1+z (esencialmente un efecto Doppler!).



Ejemplo de aplicación: Redshift fotométrico

- En el caso de datos fotométricos, solo tenemos el brillo promedio en un amplio rango de longitudes de onda, llamadas filtros o bandas.
- Mucho más desafiante/menos preciso, pero mucho más barato de obtener!
- El photo-z se puede derivar para miles de millones de galaxias.
- Los corrimientos al rojo espectroscópicos (derivados de la identificación de líneas) se pueden usar como un conjunto de aprendizaje para los corrimientos al rojo fotométricos.

